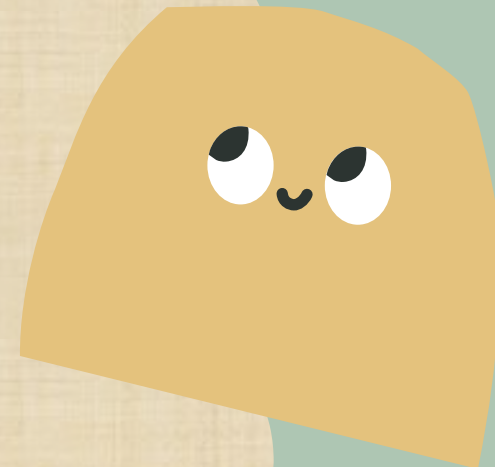
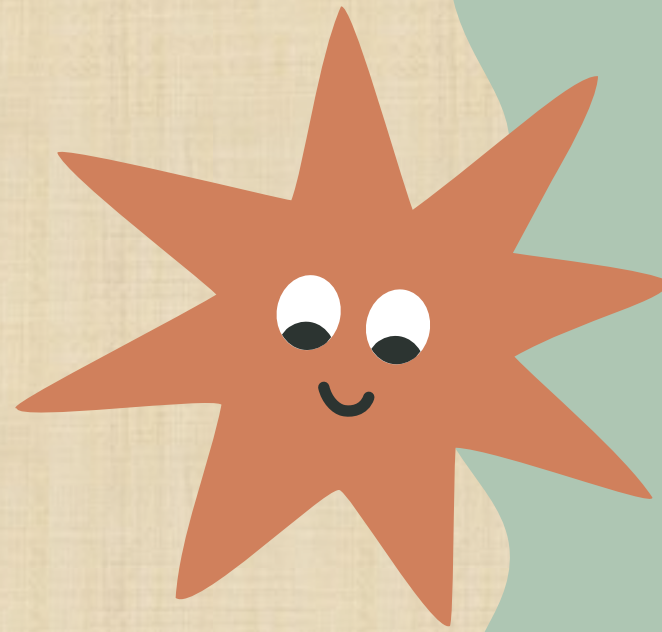


UNIT 2 BASICS OF PROGRAMMING OF C++

10TH COMPUTER SCIENCE
SINDH TEXTBOOK BOARD

<https://awinashgoswami.github.io>



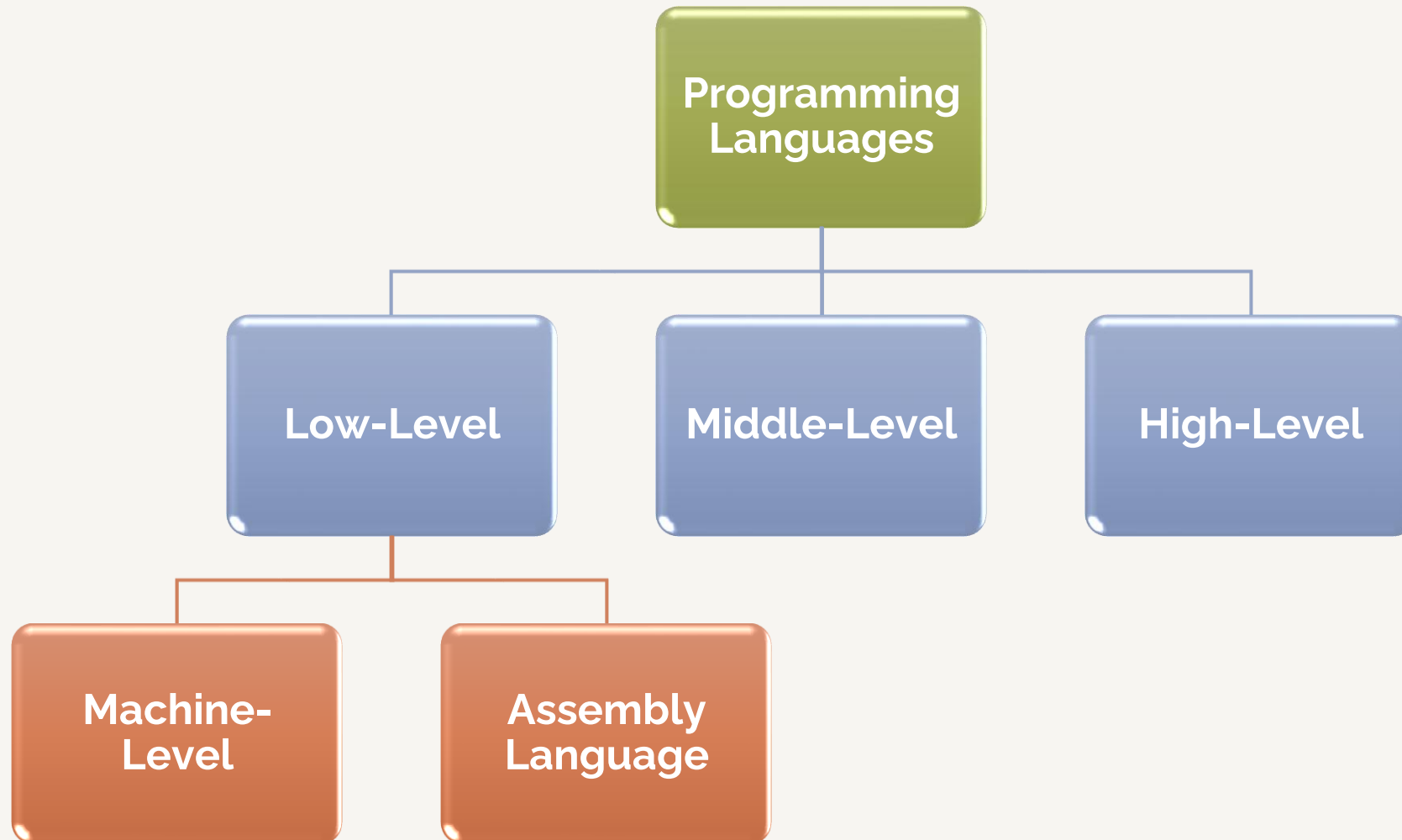
Agenda

- Introduction
- Classification of Programming languages
- Translators
- Types of Errors
- C++ Programming Language

Introduction

- A computer program (also known as software) is a set of instructions that perform a specific task. E.g.
 - MS Word (Creating Documents)
 - CapCut (Video Editing)
 - VS Code (Coding) etc.
- Different languages are used to write these software such as; C++, Java, Python, C# etc.
- Syntax is like a Grammar of Programming that follows programming rules. e.g. `cout<<"Hello World";`
- The above C++ coding example follow rules to print text – Hello World. If we remove semicolon, it rules out programming grammar which can throw an error.

Classification of Programming Languages



Low-Level Programming Language

- Low Level Language can directly communicate with the hardware.
- Types of Low-Level Language are Machine level language and Assembly language.
- *Machine-level language:*
 - Machine-level language directly accesses hardware and is written in binary (0s and 1s). It is hard for humans to understand but needs no translator, as the computer executes it directly.
- *The Assembly language:*
 - Assembly language sits above machine language, so it has less direct hardware access. Written in simple English-like words, it is easy for humans to read, write, and maintain. However, it requires an assembler to convert it into machine code.

Middle-Level Programming Language

- A middle-level programming language combines the low-level control of hardware with the readable syntax of high-level languages.
- It allows direct memory manipulation since it is near to hardware.
- Prominent examples are C, C++, and Rust.

High-Level Programming Language

- High-level languages resemble human language more than machine code (0s and 1s).
- They lack direct hardware access, so a translator is needed to convert them into low-level language.
- They are portable and machine-independent.
- Examples include Python, Java, and JavaScript.

Translators

- Computers understand only binary, while humans write code in high-level languages—known as source code. To bridge this gap, a translator is needed.
- There are three types of translators.



Compiler

Interpreter

Assembler

Compiler

- A compiler converts high-level language into low-level language all at once and creates an executable file.
- This program can be reused without recompiling from source.
- If conversion fails, it throws errors and may crash the program—errors can only be fixed by editing the source code and recompiling.
- Examples: GCC/Clang (C++), Javac (Java), tsc (TypeScript).

Interpreter

- An interpreter also converts high-level language into low-level language, but translates line by line and reports errors as soon as they occur.
- This makes error detection easier than with a compiler.
- It is faster since it executes code immediately upon reading it, but it does not create an executable file.
- Examples: CPython, PyPy, JavaScript Interpreter, PHP Interpreter.

Assembler

- An assembler is a translator used to translate assembly language to machine language.
- It is like a compiler for the assembly language but interactive like an interpreter.
- Examples of assemblers; Netwide Assembler, GNU Assembler

Compiler VS Interpreter VS Assembler

Feature	Compiler	Interpreter	Assembler
Definition	Translates high-level language to machine code	Translates high-level language to machine code	Translates assembly language to machine code
Input Language	High-level language (C, Java, C++)	High-level language (Python, JavaScript)	Assembly language (MOV, ADD, JMP)
Translation Method	Whole program at once	Line by line	Instruction by instruction
Error Reporting	After complete translation (all errors together)	Immediately when error is found (line by line)	Immediately when error is found (instruction by instruction)
Speed of Translation	Slow (takes time to compile)	Fast (starts executing immediately)	Fast (simple 1-to-1 translation)
Examples	GCC, Clang, javac	Python IDLE, Node.js	NASM, MASM, GAS
Best For	Large programs where speed matters	Small programs, scripting, debugging	Low-level hardware programming
Analogy	Reads whole book, then summarizes	Reads and explains each line instantly	Translates each word one by one

Types of Errors

- Errors are the problems that occur in the program which cause the program to behave abnormally.
- Programming errors often remain undetected until the program is compiled or executed.
- Most common programming errors can be classified in following.



**Syntax
Error**

**Run-Time
Error**

**Logic
Error**

Syntax Error

- A syntax error occurs when code violates the rules of a programming language—such as misspelled keywords, missing punctuation, brackets, or closing parentheses.
- IDEs detect these as you type, underlining faulty statements with a wavy line.
- Running a program with syntax errors shows error messages, and the program won't execute.
- Common examples:
 - missing parenthesis `()`,
 - using an undeclared variable,
 - and missing semicolon.

Run-Time Error

- Run-time errors occur during program execution after successful compilation.
- They happen when a program is asked to perform something it cannot, causing a crash—such as dividing by zero, array out of bound, allocating more memory than available, accessing unreachable server etc.
- The code has no syntax or logic errors, but fails to carry out its task when run.

Logic Error

- Logic errors prevent a program from doing what was expected.
- They produce incorrect output while appearing error-free, with no warnings at all.
- For example, a program that averages three numbers but divides their sum by 5 instead of 3 will run normally, show no error messages, and display a wrong result—a classic logical error.

C++ Programming Language

- C++, created by Bjarne Stroustrup, is a high-level general-purpose language and an upgraded version of C.
- It added object-oriented concepts missing in C.
- C++ builds both low-level software (drivers, kernels) and higher-level applications (games, GUIs, desktop apps).
- Its basic syntax and code structure match C's.

C++ Programming Language Cont....

- **Reserved Words:**

- A reserved word in C++ is a word whose meaning is already defined by the compiler which we cannot use for personal use such as identifier (also known variable), function etc.
- A reserved word is part of syntax and may not have any specific meaning in English language.
- There is a total of 95 reserved words in C++.

and	break	case
auto	char	do
bool	class	else
catch	const	export
default	continue	float
double	delete	goto
enum	explicit	inline
extern	false	module
for	friend	new
if	import	or
int	long	protected
nullptr	namespace	short
public	not	static
requires	operator	struct
signed	private	template
switch	register	throw
this	return	typedef
true	sizeof	union
unsigned	try	virtual
void	using	while

C++ Programming Language Cont....

- **Data Type:**

- Data types are like container with specific format and size to store value e.g. integer (4byte), float (4byte), Boolean(1byte), double(8byte), char(1byte) etc. for example: price is a number, we can't use characters in it.

- **Constant:**

- A constant is data item whose value cannot be changed during runtime. Const keyword is used to make data time constant. For example; `const float PI = 3.14;` The code states "Initialize a float constant named PI with value 3.14". The name of constant value is recommended in caps.

- **Variable:**

- A variable is data item whose value can be changed during runtime. For example; `int age = 25;` The code states "Initialize an integer variable named age with value 25".

C++ Programming Language Cont....

- **Rules for Naming Variables**

- Names can contain letters, digits and underscores
- Names must begin with a letter or an underscore (_)x
- Names are case sensitive (myVar and myvar are different variables)
- Names cannot contain whitespaces or special characters like !, #, %, etc.
- Reserved words (like C++ keywords, such as int) cannot be used as names
- Names cannot be longer than 32 characters in C++ by default.

- **Declaring a Variable**

- Declaring a variable mean to reserve a space for data item. For example; `int year;` The code states reserve a space of 8 bytes in memory for an integer. Compiler does not care whatever name you have kept for the variable; it only cares the number of bytes.

C++ Programming Language Cont....

- **Initializing a Variable**

- Initializing a variable mean to reserve a space for data item and put this data item in the space.. For example; `int year = 2026;` The code states reserve a space of 8 bytes in memory for an integer where I am storing value which is 2026. Compiler does not care whatever name you have kept for the variable; it only cares the number of bytes and value you have put.

- **Strings**

- A variable which can hold characters not numerical values is called string. For example; reserve memory for a string object named name, and initialize it with the text value "Ahmed". Compiler does not care whatever name you have kept for the variable; it only cares the number of bytes and value you have put.

C++ Programming Language Cont....

- Test the program in DevC++ and see output

```
1  #include<iostream>
2  using namespace std;
3  int main()
4  {
5      //declaration example: int is datatype here and age is identifier
6      int age;
7
8      //initialization example: float is datatype and PI is identifier with value 3.14
9      const float PI = 3.14;
10
11     //initialization example: string is datatype and name is identifier with value awinash
12     string name = "awinash";
13
14     //printing values
15     cout<<"age: "<<age<<endl;
16     cout<<"PI: "<<PI<<endl;
17     cout<<"name: "<<name<<endl;
18 }
```